EVALUACIÓN GLOBAL · INTEGRACIÓN Y DESPLIEGUE

Global de Programación III

Librería web integrada y desplegada en Easypanel

Objetivos

- Conectar las pantallas del primer parcial con la API de la librería mediante JavaScript nativo.
- Usar los contratos de FastAPI para consultar el catálogo y registrar libros, clientes y ventas.
- Construir una imagen Docker del frontend y publicar la aplicación en una VPS administrada con Easypanel.
- Verificar desde la URL pública que las operaciones llegan a la API y que los datos quedan en PostgreSQL.

Contexto y alcance

En el primer parcial, el navegador simula las altas y ventas. En el Global se reutilizan esas pantallas y se conectan al TP «Librería: catálogo, clientes y ventas con FastAPI y PostgreSQL», de los capítulos 1 a 4.

El frontend conserva HTML, CSS y JavaScript nativos y recursos locales. Los registros y los importes confirmados provienen de la API y se guardan en PostgreSQL. La interfaz puede mantener en memoria el borrador de una venta.

La interfaz obligatoria cubre el catálogo, las altas de libros y clientes, la venta y el historial. Usá datos ficticios en el despliegue y en las pruebas.

Qué se entrega

Una aplicación integrada, el código fuente de ambos componentes, el Dockerfile del frontend y la configuración necesaria para reproducir el despliegue. Los cinco ejercicios suman **100 puntos**.

> **Demostración mínima**
> Desde la URL desplegada, consultá el catálogo, registrá una venta con dos renglones y volvé a abrir el detalle después de recargar la página. Los importes del historial deben coincidir con los que devolvió la API.

CONSIGNAS · INTEGRACIÓN

Catálogo y altas

Ejercicio 1 · Catálogo y cliente HTTP

20 puntos

- Centralizá la URL base y las peticiones con **fetch**. La guía usa **/api** como prefijo público. Conservá la barra final de las rutas que la traen, por ejemplo **/api/libros/**.
- Cargá **GET /libros/** y usá **titulo** para la búsqueda parcial sin distinguir mayúsculas. El listado devuelve una lista. Para paginar, enviá **limit** (1 a 100, por defecto 20) y **offset** (desde 0). Al cambiar el filtro, reiniciá el desplazamiento.
- Mostrá título, ISBN, precio, editorial y género. Para ver los autores, consultá **GET /libros/{id}**. Si el género es nulo, mostrá «Sin clasificar». Diferenciá carga, lista vacía, búsqueda sin coincidencias y error de conexión.

Ejercicio 2 · Alta de libros y clientes

20 puntos

- Para **POST /libros/**, enviá título, ISBN, precio positivo, **stock** entero desde cero, editorial, género opcional y al menos un autor. Cada autoría incluye **autor_id**, **rol** y **orden**. Los roles de autoría son autor, coautor, traductor e ilustrador.
- Cargá las opciones de editorial y género desde **GET /editoriales/** y **GET /generos/**. Para seleccionar autores, podés verificar cada ID con **GET /autores/{id}** y mostrar su nombre antes de agregarlo.
- Para **POST /clientes/**, enviá **nombre** y **email**. Si el backend solo expone la consulta individual, usá **GET /clientes/{id}** para buscar clientes y mostrar su nombre antes de seleccionarlos para una venta.
- Validá los campos y mostrá el resultado cerca del formulario. Un alta exitosa devuelve **201**. Usá el registro devuelto para actualizar las vistas relacionadas. Mostrá el mensaje del servidor ante un **409** o **422**.

> **Consultas por ID**
> El contrato de referencia ofrece consultas individuales de autores y clientes. Si tu API también tiene listados o buscadores, podés usarlos para los selectores. Documentá qué opción elegiste.

CONSIGNAS · INTEGRACIÓN

Ventas e historial

Ejercicio 3 · Registrar y consultar ventas

25 puntos

Armá la venta. Seleccioná un cliente y agregá libros del catálogo. Permití cambiar cantidades y quitar renglones. Para confirmar, exigí al menos un renglón, cantidades enteras positivas y un solo renglón por libro.

Enviá la operación. Usá **POST /ventas/** con el identificador del cliente y los datos de cada renglón. El cuerpo lleva únicamente estos campos, con los identificadores que correspondan a los datos de tu base.

```
{
  "cliente_id": 2,
  "renglones": [
    {"libro_id": 1, "cantidad": 2},
    {"libro_id": 4, "cantidad": 1}
  ]
}
```

Confirmá con la respuesta. El backend devuelve **201** y la venta registrada. Mostrá los precios unitarios que copió del catálogo, los subtotales y el total de la respuesta. Limpiá el formulario después de confirmar y actualizá el historial. Deshabilitá el botón mientras se procesa el envío.

Consultá el historial. Usá **GET /ventas/** con **limit** y **offset**. Abrí **GET /ventas/{id}** para mostrar fecha, cliente, renglones e importes históricos. Un cambio posterior en el precio de un libro conserva los importes de las ventas ya registradas.

Mostrá los errores. Si el cliente o el libro dejó de existir, el servidor devuelve **409**. Un cuerpo inválido, como una venta con libros repetidos, devuelve **422**. Conservá el borrador cuando se rechaza la operación.

> **Verificación de la persistencia**
> Recargá la página y volvé a abrir la venta desde el historial. La API debe devolver el mismo detalle. El stock es un dato del catálogo. En este TP, el alta de una venta registra sus renglones e importes.

GUÍA DE DESPLIEGUE · 1 DE 5

Crear la imagen del frontend

Ejercicio 4 · Dockerfile del frontend

15 puntos

Con Docker iniciado, entrá en **frontend/** y ejecutá **docker info** para verificar que Docker responde.

1. Separá los archivos públicos

Creá **public/** dentro de **frontend/**. Mové ahí index.html, las demás páginas y las carpetas de CSS, JavaScript e imágenes. Conservá sus rutas relativas y revisá los enlaces que apuntaban a archivos fuera de esa carpeta.

```
frontend/
  Dockerfile
  .dockerignore
  default.conf.template
  public/
    index.html
    css/, js/, img/
```

Guardá la configuración y los archivos de entorno fuera de **public/**.

2. Creá Dockerfile y .dockerignore

Creá un archivo llamado **Dockerfile** sin extensión y pegá estas líneas. En el paso 3 vas a crear la plantilla que usa el segundo COPY.

```
FROM nginx:1.30-alpine
ENV NGINX_ENVSUBST_FILTER=^API_UPSTREAM$
COPY public/ /usr/share/nginx/html/
COPY default.conf.template /etc/nginx/templates/default.conf.template
EXPOSE 80
```

FROM usa la imagen oficial de Nginx. El primer **COPY** coloca la web donde Nginx la sirve. El segundo agrega la plantilla del proxy. **EXPOSE 80** declara el puerto interno. La publicación del puerto se configura al ejecutar el contenedor.

Creá **.dockerignore** junto al Dockerfile para excluir estos archivos del build.

```
.git
**/.env
**/.env.*
**/node_modules
**/.DS_Store
```

Imagen de referencia · Nginx en Docker Hub

GUÍA DE DESPLIEGUE · 2 DE 5

Conectar y probar en local

3. Creá la plantilla de Nginx

Guardá este contenido como **default.conf.template** junto al Dockerfile. Nginx sustituye **API_UPSTREAM** al iniciar el contenedor. Su valor tiene la forma **host:puerto**, sin **http://** ni barra final.

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://${API_UPSTREAM}/;
    }
}
```

La barra final de **proxy_pass** hace que **/api/libros/** llegue a FastAPI como **/libros/**. En el cliente HTTP, usá **/api** como base y sumale la ruta exacta del backend. Por ejemplo, **fetch("/api/libros/")**. La dirección interna se configura en Nginx.

4. Construí y ejecutá la imagen

Con el entorno Python activo y **DATABASE_URL** configurada para la base local, aplicá las migraciones y levantá el backend desde su carpeta. Cargá datos de prueba según el README del TP.

```
alembic upgrade head
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Desde **frontend/**, ejecutá estos comandos. En Docker Desktop omití la línea **--add-host**. En PowerShell, poné el segundo comando en una línea y quitá las barras invertidas.

```
docker build -t libreria-frontend .
docker run --rm --name libreria-frontend \
  -p 127.0.0.1:8080:80 \
  --add-host=host.docker.internal:host-gateway \
  -e API_UPSTREAM=host.docker.internal:8000 libreria-frontend
```

Abrí **http://127.0.0.1:8080** y comprobá que **/api/health/ready** responde 200 bajo ese origen. Probá el catálogo y abrí el detalle de un libro. Detené el contenedor con **Ctrl+C**.

Referencia · Cómo Nginx reemplaza el prefijo de una ruta

GUÍA DE DESPLIEGUE · 3 DE 5

Preparar la imagen del backend

Ejercicio 5 · Servicios en Easypanel

20 puntos

La guía usa dos servicios App del mismo proyecto de Easypanel. Nginx sirve el frontend y reenvía **/api/** al backend por la red interna. El backend se conecta a la base PostgreSQL asignada.

5. Prepará el código para desplegar

Verificá que **backend/** incluye el código del TP, **requirements.txt**, **alembic.ini**, **migrations/** y **env.example**. Si el backend ya tiene Dockerfile, revisá que instale las dependencias, tenga acceso a las migraciones y escuche en **0.0.0.0:8000**.

Si tu backend todavía no tiene Dockerfile, guardá este ejemplo en **backend/Dockerfile**. El comando aplica las migraciones antes de iniciar la API.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD alembic upgrade head && \
    exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Junto a ese archivo, creá **backend/.dockerignore** para excluir el entorno local, los metadatos de Git y las credenciales.

```
.git
**/.env
**/.env.*
**/.venv
**/__pycache__
**/.pytest_cache
```

Subí el código a Git con los Dockerfiles, los archivos de configuración y la plantilla de Nginx. Conservá **env.example** con valores de muestra. Cargá los valores reales en el entorno privado del servicio.

GUÍA DE DESPLIEGUE · 4 DE 5

Desplegar la API en la VPS

6. Creá y desplegá el servicio backend

- Dentro del proyecto de tu grupo, elegí **New Service** → **App** y asignale el nombre **backend**.
- En **Source**, seleccioná el repositorio, la rama y la carpeta que contiene el backend. Para un repositorio con ambos componentes, usá **/backend** como **Build Path**. En **Build**, elegí Dockerfile y escribí **Dockerfile** como ruta relativa.
- Usá la base PostgreSQL asignada. Si es un servicio Postgres de Easypanel, consultá el host y puerto internos en **Credentials**. En **Environment** del backend, cargá **DATABASE_URL** con el driver **postgresql+asyncpg** y los datos de esa base. Mantené el puerto de PostgreSQL privado.
- Revisá que la imagen de la API arranque Uvicorn en **0.0.0.0:8000**. Pulsá **Deploy** y revisá el build y los logs. Si tu imagen aplica **alembic upgrade head** al arrancar, confirmalo en los logs. Si el contenedor ya corre y no lo aplica, ejecutalo desde **Shell** antes de probar la API.
- En **Shell**, comprobá la salud con el comando de abajo. Después ejecutá el script o comando de carga inicial documentado en tu backend para preparar los datos ficticios. Revisá qué tablas modifica antes de ejecutarlo sobre una base existente.

```
URL=http://127.0.0.1:8000/health/ready
python -c "from urllib.request import urlopen; print(urlopen('$URL').status)"
```

El comando debe imprimir **200**. Antes de publicar el frontend, comprobá que el catálogo tenga datos. Conservá el backend en la red interna y quitá su dominio automático si Easypanel le asignó uno.

Documentación de Easypanel · Servicio App · Servicio Postgres

GUÍA DE DESPLIEGUE · 5 DE 5

Publicar y comprobar

7. Configurá el servicio frontend

- En el mismo proyecto, creá otro servicio **App** llamado **frontend**. En **Source**, seleccioná el repositorio y la rama. En un repositorio con ambas carpetas, usá **/frontend** como **Build Path**. Seleccioná Dockerfile como builder y **Dockerfile** como ruta relativa.
- En **Environment**, configurá **API_UPSTREAM** con el nombre interno del backend y su puerto. Por ejemplo, para el proyecto **grupo1** y el servicio **backend**, usá el valor de abajo. Reemplazá **grupo1_backend** por el nombre interno de tu instalación.

```
API_UPSTREAM=grupo1_backend:8000
```

- En **Domains**, usá el dominio HTTPS asignado al frontend. Configurá **HTTP** como protocolo interno y **80** como puerto de destino. Easypanel recibe HTTPS y se comunica por HTTP con Nginx dentro de la VPS.
- Guardá los cambios y pulsá **Deploy**. Abrí el dominio público. Después agregá **/api/health/ready** a esa URL y verificá que responde **200**. Probá el catálogo y registrá una venta de prueba desde esa misma URL.

8. Diagnosticá y dejá evidencia

Si ves esto	Revisá esto
Falla un COPY	El Build Path, los nombres exactos de los archivos y la carpeta public/.
Nginx no arranca o /api/ devuelve 502	Los logs del frontend, API_UPSTREAM, el nombre interno, el puerto y si el backend está en ejecución.
/api/health/ready falla	Los logs del backend, DATABASE_URL, la conexión a PostgreSQL y las migraciones.
Una petición de la API devuelve 404	En Red del navegador, verificá el prefijo /api y la ruta del backend, incluida su barra final cuando corresponda.

Registrá la URL, los puertos, los pasos de despliegue y los resultados en **DEPLOY.md**. Ejecutá los casos G-01 a G-07. Al redesplegar, comprobá que los datos siguen disponibles.

Documentación de Easypanel · Configuración del build · Dominio, variables y logs

VERIFICACIÓN Y ENTREGA

Casos de prueba y entrega

Ejecutar los casos desde la URL pública con datos ficticios. Registrar pasos, resultado observado y evidencia en el README o en la carpeta **evidencia/**. Ocultar contraseñas y valores privados en las capturas.

Caso	Resultado esperado
G-01 · Catálogo	Buscar por título, avanzar con limit y offset y probar una búsqueda sin coincidencias. Abrir el detalle de un libro.
G-02 · Alta de libro	Crear un libro con stock, género opcional y autorías. Consultar el detalle. Repetir el ISBN y verificar el 409.
G-03 · Alta de cliente	Crear un cliente y recuperarlo por ID. Repetir el email y verificar el 409.
G-04 · Venta	Registrar una venta válida con dos renglones. Verificar 201 y consultar el detalle después de recargar.
G-05 · Precio histórico	Cambiar el precio con **PATCH /libros/{id}** y verificar que la venta anterior conserva sus importes.
G-06 · Rechazo	Probar un ID inexistente (409) y renglones repetidos (422). El historial queda igual.
G-07 · Despliegue	La imagen del frontend se construye. La URL pública abre por HTTPS, la API devuelve 200 en salud y los datos siguen tras un redespliegue.

Criterios de evaluación

Catálogo y cliente HTTP **20** · Alta de libros y clientes **20** · Ventas e historial **25** · Imagen del frontend **15** · Despliegue y verificación **20**. Se puntúa el funcionamiento observado y su reproducción con la entrega.

Formato de entrega

Un ZIP por grupo con **frontend/**, **backend/**, **README.md**, **DEPLOY.md** y **evidencia/**. Adjuntar los Dockerfiles, la configuración versionada del servidor web y un archivo de ejemplo con los nombres de las variables. Excluir archivos .env, credenciales reales y dependencias instaladas.

El README identifica al grupo y explica cómo ejecutar la app localmente y cargar los datos de prueba. En DEPLOY.md registrar URL pública, revisión, rutas de build, puertos, proxy, nombres de variables, migraciones, carga inicial y pasos de actualización.

La cátedra comunica por separado el canal y la fecha. Mantener la URL operativa durante la corrección.